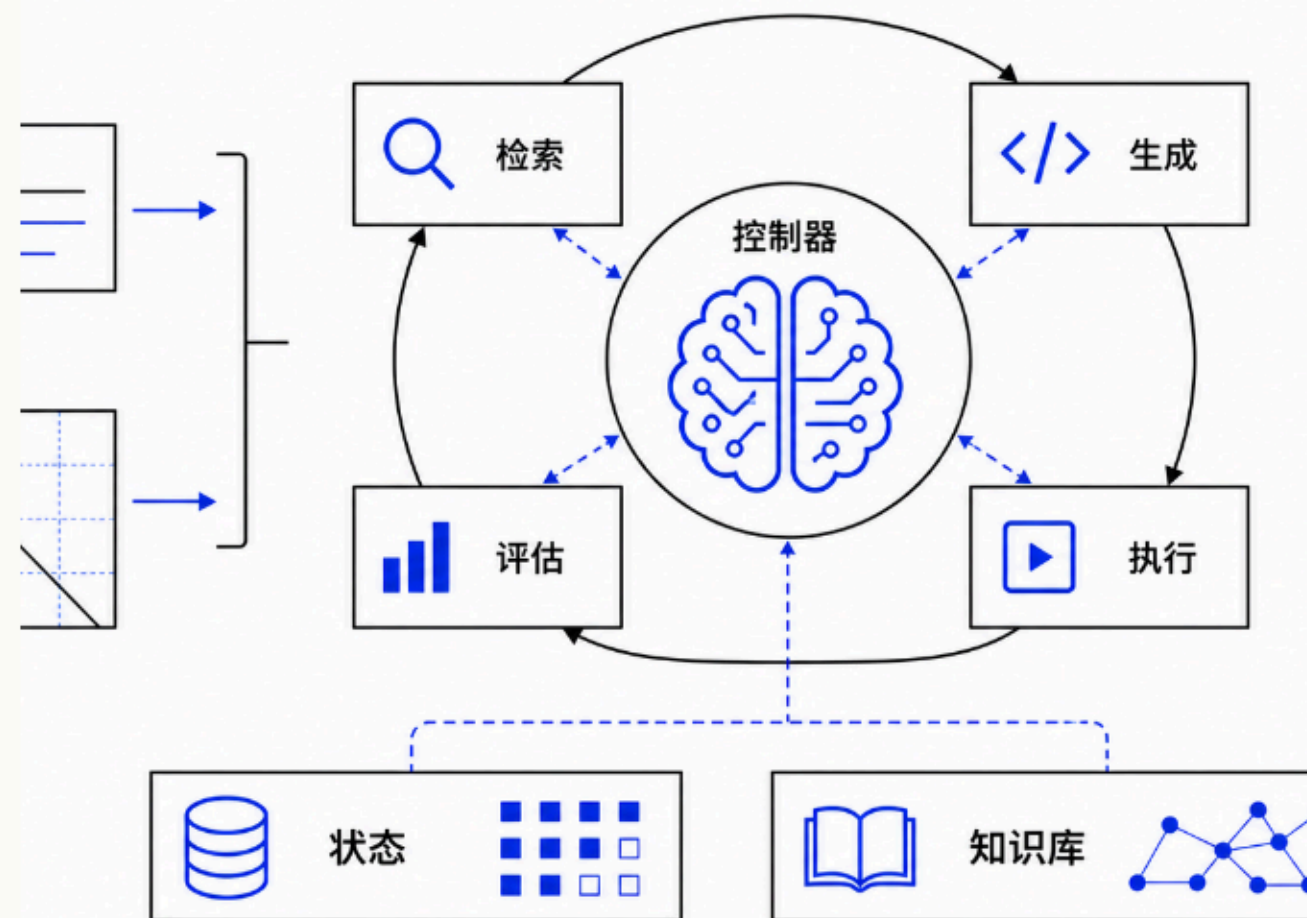


PROJECT REPORT

图像算法 自动化 Agent

用户输入提示词与图片，系统自动完成规划、检索、代码生成、本地执行、质量评估和多轮迭代。核心不是固定流程，而是由 ControllerAgent 驱动的可观测闭环。



SYSTEM BOUNDARY

从“图像处理功能” 升级为 “图像算法任务代理”

1

输入：自然语言需求 + 可选图片上传

4

核心 Agent：检索、生成、执行、评估

3/5

标准 / 质量策略下的最大自动迭代轮数

SSE

状态、消息、完成事件实时推送到前端

用户请求 文本提示词、图片上传、会话上下文。	策略规划 TaskParser 识别任务类型，PlanningEngine 决定执行路径。	Agent 执行 按计划调用检索、代码生成、执行、评估。	闭环迭代 评分和错误诊断成为下一轮输入。	状态持久化 state.json、state_logs.jsonl、agent_calls.jsonl。	前端可视化 聊天窗口和 ProcessTimeline 展示过程。
--	---	--	--	--	---

CONTROLLER AS BRAIN

ControllerAgent 是决策大脑

它维护 session、解析任务、选择策略、调度 Agent、记录调用、处理迭代结果，并把状态事件推送给前端。

任务理解

TaskParser 将自然语言映射为 simple / medium / complex、约束和置信度。

策略规划

PlanningEngine 在 fast / standard / quality / conservative 中选择计划。

过程控制

评分、错误、最大迭代次数共同决定继续、停止或人工审查。

协作信号

CollaborationCoordinator 转发检索质量、错误反馈、前置评估。

状态管理

状态快照、日志和 Agent 调用记录落盘，支持恢复和审计。

学习复用

KnowledgeBase 推荐历史成功方案，提升类似任务启动质量。

CORE AGENTS

四个 Agent 形成任务闭环，每个 Agent 有清晰责任边界

RetrievalAgent

- LLM 规划搜索词
- Tavily 多路检索
- 综合算法简报
- 质量自评 + TTL 缓存

CodeGenerationAgent

- OpenRouter 生成 Python 代码
- 注入本地库约束
- 接收迭代反馈
- 支持错误修复模式

ExecutionAgent

- 准备执行环境
- 执行 generated code
- 检查输出图片
- 诊断异常类型

EvaluationAgent

- 多模态图片评估
- 代码评估回退
- 提取 SCORE
- 输出可执行改进建议

同一个入口，根据任务复杂度动态选择不同 Agent 序列

策略	触发条件	Agent 序列	检索	评估	迭代
fast	simple 或 speed	CodeGen → Execution	skip	skip	1
standard	medium	Retrieval → CodeGen → Execution → Evaluation	on	on	3
quality	complex	完整链路 + 更多优化机会	on	on	5
conservative	低置信度	完整链路	on	on	3

```
TaskParser.parse(user_request) → {"task_type", "keywords", "constraint", "confidence"}
PlanningEngine.plan(task) → {"strategy", "agent_sequence", "skip_steps", "enable_iteration", "max_iterations"}
```

AGENTIC SEARCH

检索不是固定模板拼接，而是“规划 → 搜索 → 综合 → 自评”

RetrievalAgent 先让 LLM 从算法原理、Python/OpenCV 实现、最佳实践三个角度规划搜索词；再用 Tavily 检索；最后让 LLM 综合成代码生成简报，并输出质量分与 should_skip。

Plan Queries

LLM 生成 2-3 个英文搜索查询。

Tavily Search

advanced search，多路获取 answer 与结果。

TTL Cache

1 小时缓存，降低重复搜索成本。

Synthesis

提炼推荐算法、实现要点、代码片段、潜在问题。

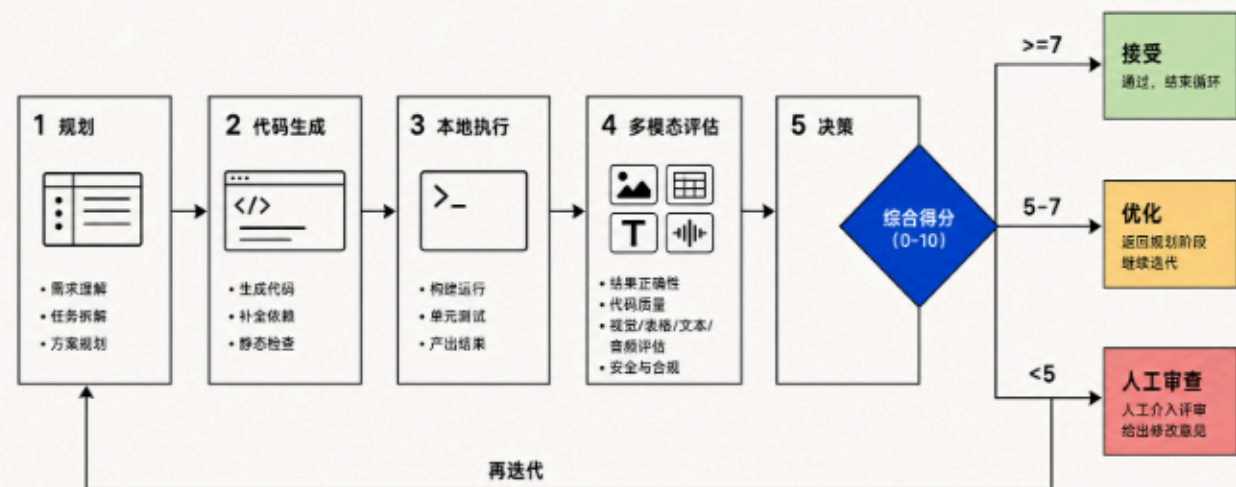
Quality Gate

1-10 分质量自评，低质资料降低依赖。

SCORE DRIVEN LOOP

评分不是展示项，而是下一步决策信号

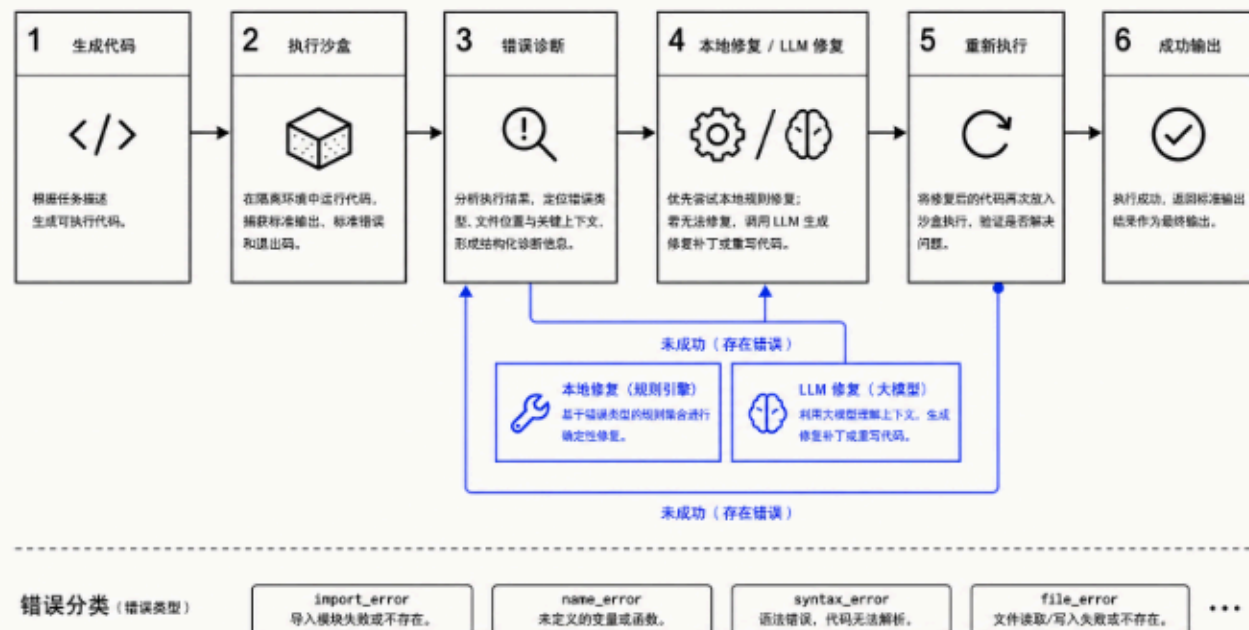
EvaluationAgent 要求返回机器可解析的 `SCORE: X/10`。Controller 将 score 与 max_iterations 结合，决定接受、继续优化、停止或转人工审查。



EXECUTION REPAIR

执行失败不会直接退出，而是形成自动修复链路

ExecutionAgent 捕获 traceback，分类为 `import_error`、`name_error`、`attribute_error`、`type_error`、`file_error`、`syntax_error` 等，再由 CodeGenerationAgent 优先本地修复，必要时 LLM 修复并重新执行。



SESSION RESOURCES + KNOWLEDGE BASE

状态管理让过程可恢复；知识库让历史结果可复用

```
sessions/<session_id>/
  uploads/          # 用户上传图片
  outputs/          # 每轮生成图片
  workspace/        # iteration_N.py
  state.json         # 会话快照
  state_logs.jsonl   # 状态流
  agent_calls.jsonl  # Agent 调用审计
sessions/latest → 最近会话
```

问题模式

通过关键词抽取与 Jaccard 相似度匹配历史问题。

成功方案

记录 approach、score、success_count、code_samples。

权重更新

成功提升复用权重，失败降低方案评分并可标记 inactive。

REALTIME PRODUCT SURFACE

前端不是只等结果，而是实时呈现 Agent 思考与执行轨迹

InputArea

提交文本和图片，创建或复用 session。

api.ts

先打开 `/api/stream`，再 POST `/api/process`。

EventSource

处理 message / state / status / error / complete。

Polling Fallback

SSE 失败时轮询 `/api/session/:id`。

Zustand Store

维护 sessions、messages、stateLogs、sessionStatus。

Timeline

ProcessTimeline 展示各 Agent 状态流转。

消息层

ChatWindow + MessageItem 渲染用户输入、agent_update 和最终回复。

过程层

stateLogs 让检索、生成、执行、评估每一步都有可见证据。

会话层

Sidebar 管理历史会话，hydrateSessionFromBackend 支持重启恢复。

AGENTIC EVIDENCE MATRIX

当前实现已经覆盖 agentic 系统的五个核心特征

能力	项目实施	关键文件	信号	价值	状态
计划	任务分类 + 策略库	task_parser.py / planning_engine.py	strategy, agent_sequence	避免一刀切流程	已实现
执行	生成代码并本地运行	code_generation_agent.py / execution_agent.py	output_path, execution_log	把算法落到结果图	已实现
感知	多模态评估 + 结构化评分	evaluation_agent.py	SCORE, improvements	把质量变成决策依据	已实现
迭代	评分阈值 + 最大轮数控制	controller.py	best_result, final_score	自动优化且避免无限循环	已实现
状态	session 资源、状态日志、调用日志	session_resources.py	json/jsonl	可恢复、可追踪、可审计	已实现

FROM MVP TO PRODUCT

可靠性
可观测性
可扩展性

算法能力

沉淀更多高质量模板、参数搜索策略和图像任务基准。

执行安全

增强代码沙箱、资源限制、文件访问约束和可解释错误报告。

产品体验

把每轮结果、评分、代码 diff 和用户反馈合并为可操作工作台。